

Weather Station Simulator - Final Report

Course: Communications Lab II

Name: GEORGIOS PAPAACHARIOU

Student ID (AM): 1092681

Date: 17-01-2026

1. Overview

This system implements a distributed Weather Station Simulator where multiple Dockerized station clients periodically generate sensor measurements and send them as newline-delimited JSON batches over TCP to a central async server. The server handles multiple concurrent connections, validates each batch (schema, timestamp format, numeric ranges), and stores accepted readings persistently in an SQLite database. A Flask-based web dashboard reads the same database to display available stations, summary statistics, and time-series charts for selected metrics. In addition to producer stations, the TCP server supports simple read-only consumer queries using JSON request messages over the same socket interface.

2. Architecture

The system is composed of three main components deployed with Docker Compose: station clients, a central TCP server, and a web dashboard. Station clients run as separate Docker containers and periodically generate weather measurements (e.g., temperature, humidity, windspeed). They transmit these measurements as newline-delimited JSON batches over TCP to the server on port 12345.

The TCP server is implemented with async to support multiple simultaneous connections. For each incoming message, it performs validation (message type, required fields, timestamp format, numeric sanity/ranges). Valid readings are stored in an SQLite database mounted on a persistent Docker volume. The same TCP port also supports read-only consumer requests, allowing external clients to query stored data (e.g., list stations, latest reading, recent readings) using JSON request messages.

For visualization, a Flask + JavaScript web dashboard reads from the SQLite database (read-only) and exposes an HTTP interface on port 8000. It provides station/metric selection and renders charts and summary statistics based on the stored measurements.

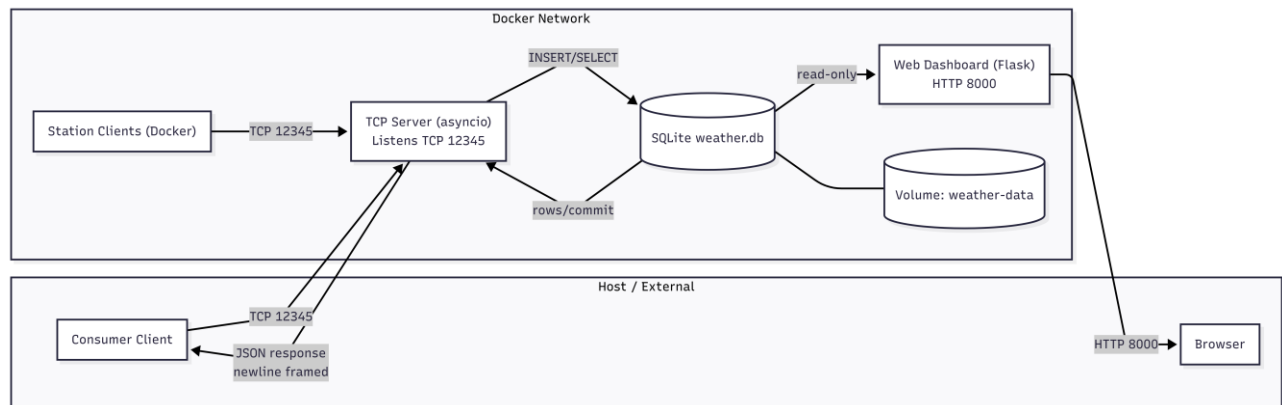


Figure 1: Component view and communication paths (TCP 12345, HTTP 8000).

3. Protocol and Message Flow

All station clients and consumer clients connect to the server over TCP (port 12345). Messages are framed as newline-delimited JSON (NDJSON), meaning each request or batch is a single JSON value terminated by “\n”, so the server can reliably parse complete messages from the TCP stream.

Two message types share the same port:

- **Producer (station) messages:** a JSON array (batch) of readings. The server validates JSON format, required fields, timestamps, and value ranges; valid batches are enqueued and inserted into SQLite, and the server replies with `{"status":"ok","inserted":N}`.
- **Consumer (query) messages:** a JSON object containing "request" (e.g., stations, latest, recent). The server runs the corresponding SQLite query and returns `{"status":"ok", ...}` (or an error for invalid/unknown requests).

The complete end-to-end flow (including validation, write queue, DB transaction, and error paths) is shown in the sequence diagram.

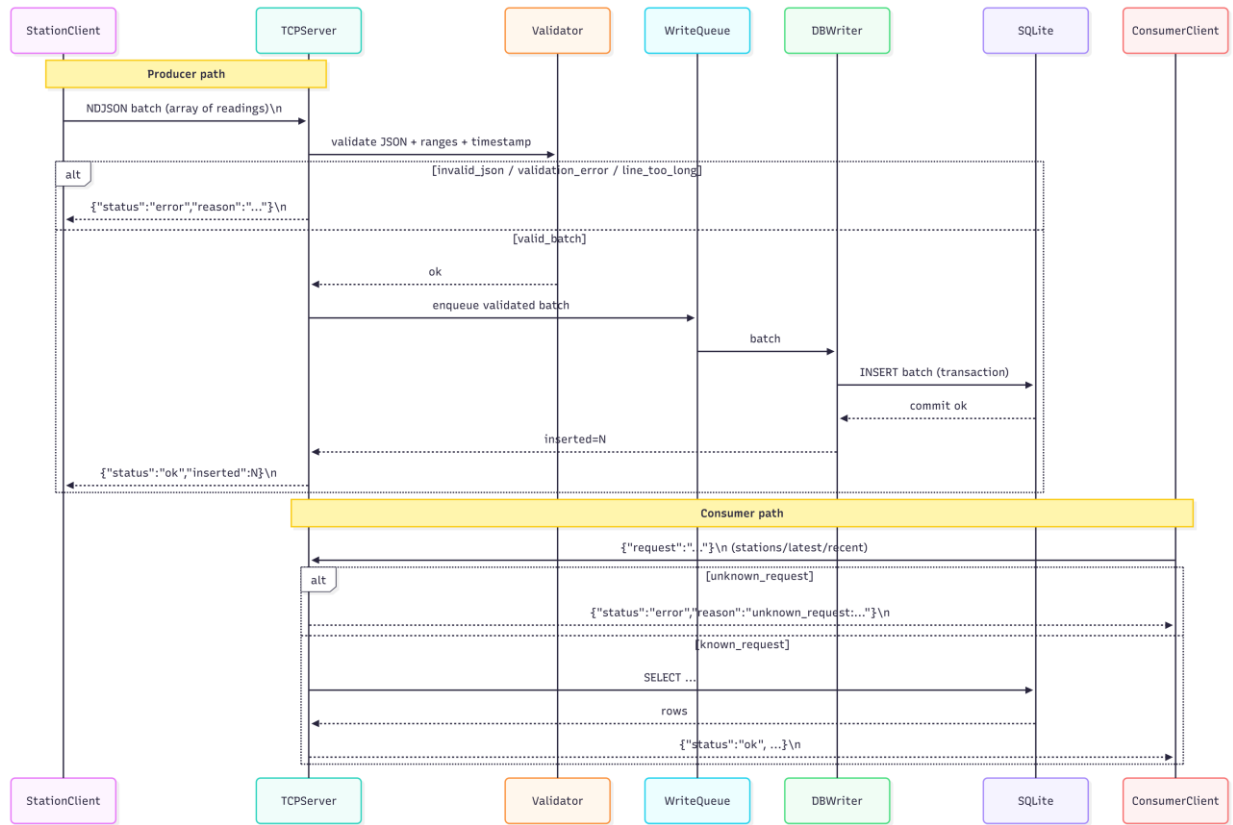


Figure 1: Sequence diagram of TCP message flow

4. Key Design Decisions

- Asyncio TCP server for concurrency: We use an asyncio server so one process can handle many station connections efficiently without a thread-per-client model, trading a bit of implementation complexity for scalability.
- Batch sending (stations): Stations send readings in small batches to reduce TCP/JSON overhead and DB transaction cost, trading a small delay (until the batch is ready) for better throughput and fewer writes.
- Strict validation: Required fields, ISO timestamps, finite numbers, and realistic ranges are enforced before insert to keep the database consistent and safe, trading flexibility for clean data.
- Single DB-writer queue + WAL + busy timeout: All inserts go through one writer task and WAL mode to avoid SQLite write contention (“database is locked”), trading maximum write parallelism for stability and predictable behavior.
- Client resiliency: Clients reconnect with backoff and buffer offline batches, then flush after reconnect to tolerate downtime, trading extra client logic/memory for robustness.
- Separation of concerns: The server is the only DB writer, while the dashboard reads SQLite (read-only), reducing contention and keeping roles clear, trading push-based updates for a simpler architecture.

5. Web Dashboard

The web UI is a lightweight Flask + JavaScript dashboard that reads the stored measurements from SQLite (read-only) and serves the interface over HTTP on port 8000. It is designed to verify that data is being ingested correctly and to provide quick visualization per station and metric.

The backend exposes three endpoints used by the frontend:

- GET /api/stations — list available station IDs
- GET /api/stats?station_id=...&metric=...&range=... (or limit=...) — summary stats (latest/min/max/avg)
- GET /api/readings?station_id=...&metric=...&range=... (or limit=...) — time-series points for charting

In the browser, the user selects station + metric + time window/last-N, and the page updates the displayed statistics and chart accordingly (optionally with a rolling average overlay).

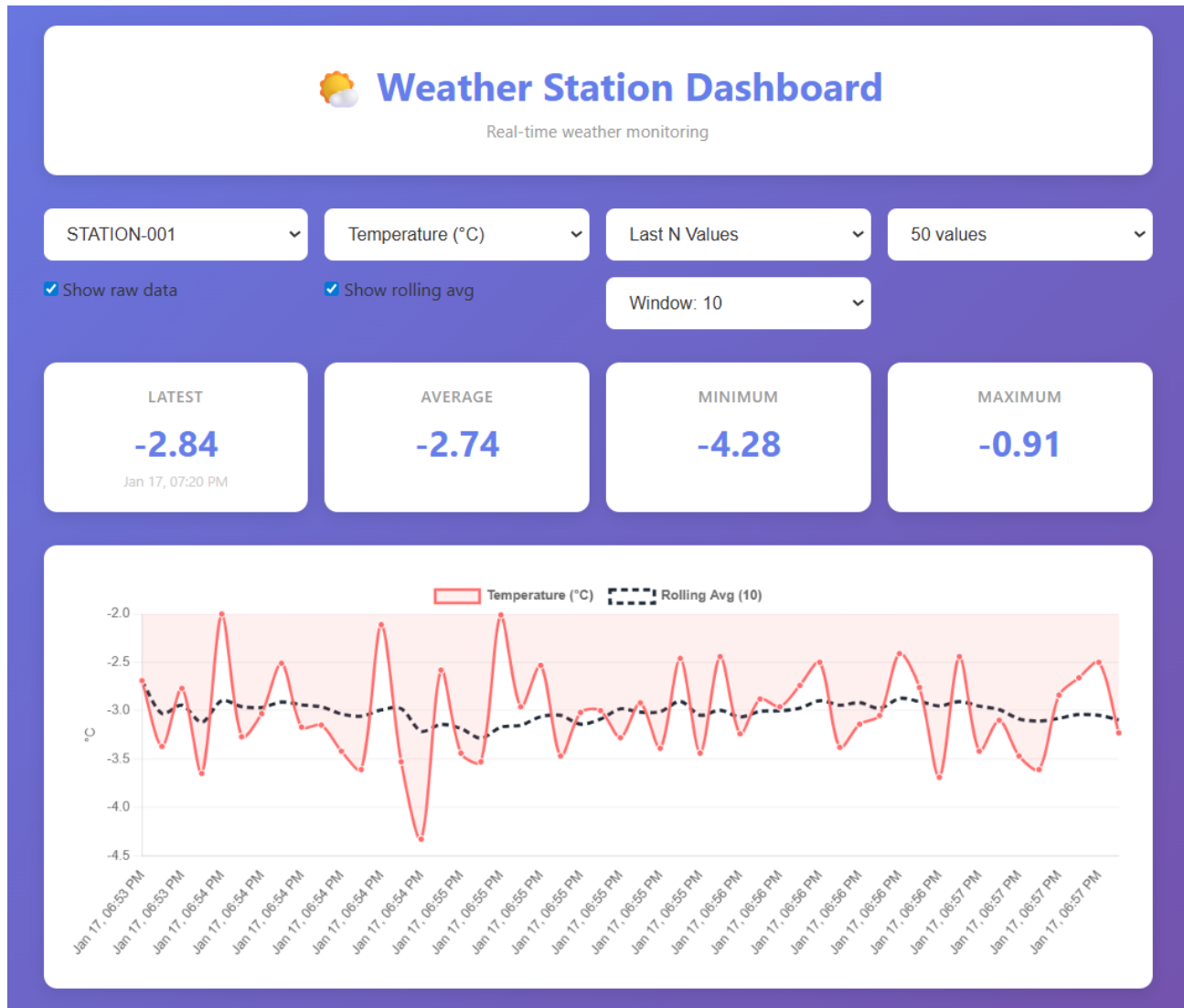


Figure X: Web dashboard visualization.

6. Testing & Results

To verify correct operation end-to-end, the system was executed with multiple station containers sending batches concurrently to the TCP server and storing data in SQLite. The server logs confirm successful batch inserts from different station clients, and the web dashboard displays the stored readings and statistics per station/metric. In addition, consumer-style TCP queries were tested to confirm the server can return stored data on request.

```
[+] Running 12/12
✓ weather-station-server          Built          0.0s
✓ weather-station-station-001     Built          0.0s
✓ weather-station-station-002     Built          0.0s
✓ weather-station-station-003     Built          0.0s
✓ weather-station-web             Built          0.0s
✓ Network weather-station_weather-network Created        0.0s
✓ Volume weather-station_weather-data Created        0.0s
✓ Container weather-server         Healthy        5.4s
✓ Container weather-station-002    Started        5.5s
✓ Container weather-web            Started        0.5s
✓ Container weather-station-001    Started        5.5s
✓ Container weather-station-003    Started        5.5s
```

Figure X: Docker Compose deployment status — All services were built and started successfully (TCP server, web dashboard, and three station clients), with the shared Docker network and persistent SQLite volume created; the server healthcheck reports Healthy.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
648a2bc6eb95	weather-station-web	"python web.py"	10 seconds ago	Up 10 seconds	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	weather-web
6258df70bd71	weather-station-station-003	"python client.py"	10 seconds ago	Up 5 seconds		weather-station-003
504c05696f3a	weather-station-station-001	"python client.py"	10 seconds ago	Up 5 seconds		weather-station-001
0d98dba15870	weather-station-station-002	"python client.py"	10 seconds ago	Up 5 seconds		weather-station-002
7f40b79b9397	weather-station-server	"python app.py"	10 seconds ago	Up 10 seconds (healthy)	0.0.0.0:12345->12345/tcp, [::]:12345->12345/tcp	weather-server

Figure X: All services running (server, web dashboard, and multiple station clients).

```
Database initialized: /app/data/weather.db
Weather Station Server running on 0.0.0.0:12345
Waiting for client connections...
(Ctrl+C to shutdown gracefully)
Database writer task started
SQLite pragmas applied: WAL mode, NORMAL sync, 3000ms busy timeout
Client connected: ('127.0.0.1', 36582)
Client disconnected: ('127.0.0.1', 36582)
Client connected: ('172.18.0.5', 47046)
Client connected: ('172.18.0.6', 33884)
Client connected: ('172.18.0.4', 40952)
```

Figure X: Init Logs.

7. Conclusion

This project delivers a complete weather-station pipeline: multiple Dockerized station clients send NDJSON batches over TCP to an asyncio server, which validates and stores readings persistently in SQLite, while a web dashboard visualizes the collected data over HTTP. The chosen design (batching, strict validation, a single DB-writer queue with WAL, and resilient clients) prioritizes correctness and stable operation under concurrency. Future improvements could include supporting dynamic/variable metric sets per station, adding authentication for untrusted networks, and extending analytics/aggregation endpoints.